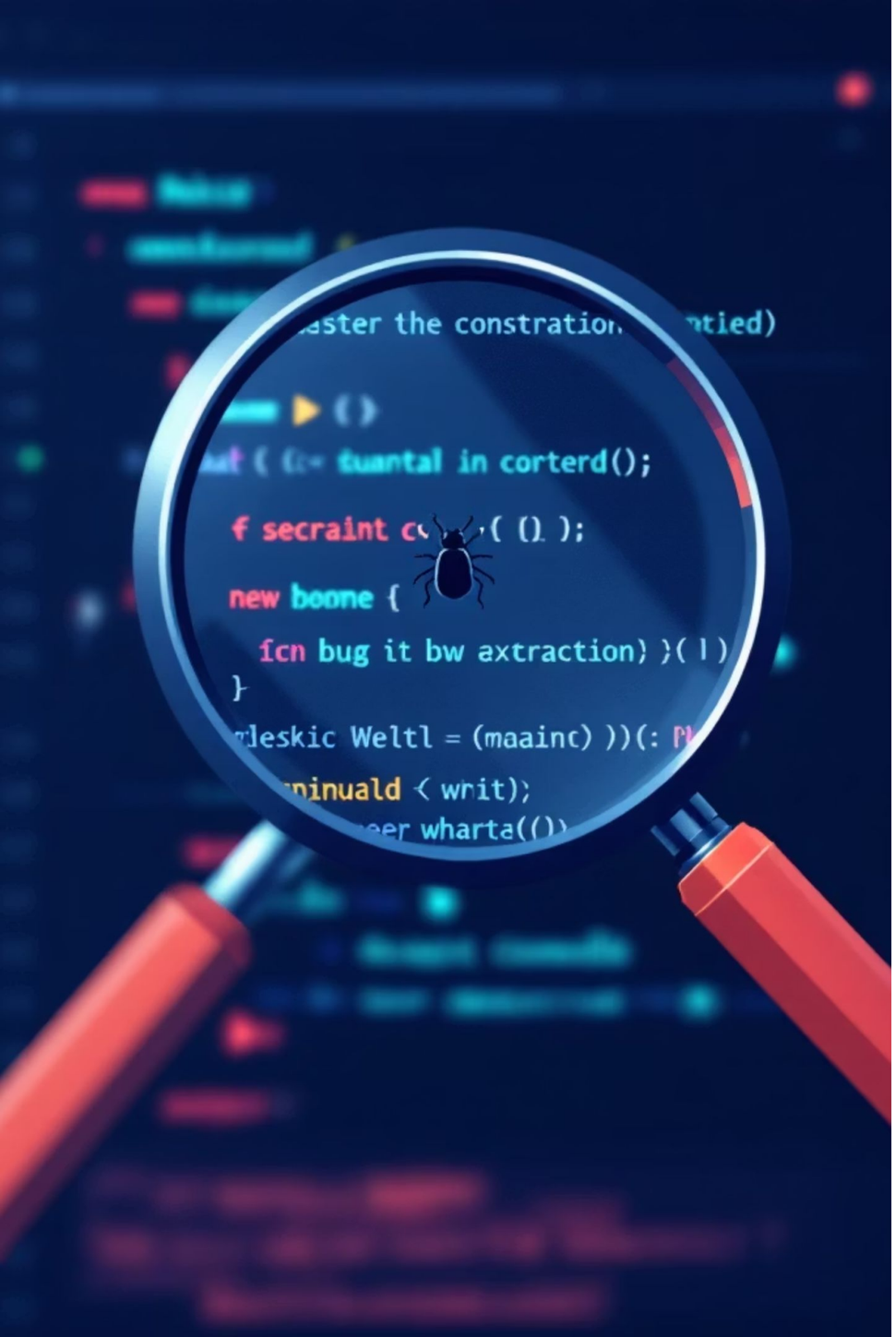
Debugging with Visual Studio Code

Topics covered:

What a debugger is, why it matters, core concepts, and how to use VS Code's debugger to find and fix bugs quickly.



What is Debugging?

Debugging is the process of finding, analyzing, and fixing errors in software. It moves beyond error messages — it reveals program state, execution flow, and unintended behavior so you can pinpoint the root cause.

- ## The Analogy

  Normal Code: Like watching a car race at full speed. You see the start and the finish, but not the engine problems in between.

  Debugging: Like putting the car on a lift and running it slowly to see exactly which gear is grinding.

Made with GAMMA

Why Debugging matters?

1. Improves software quality and reliability
2. Faster bug detection and resolution
3. Saves development time and cost
4. Helps understand program flow
5. Allows state and variable inspection
6. Essential for complex systems

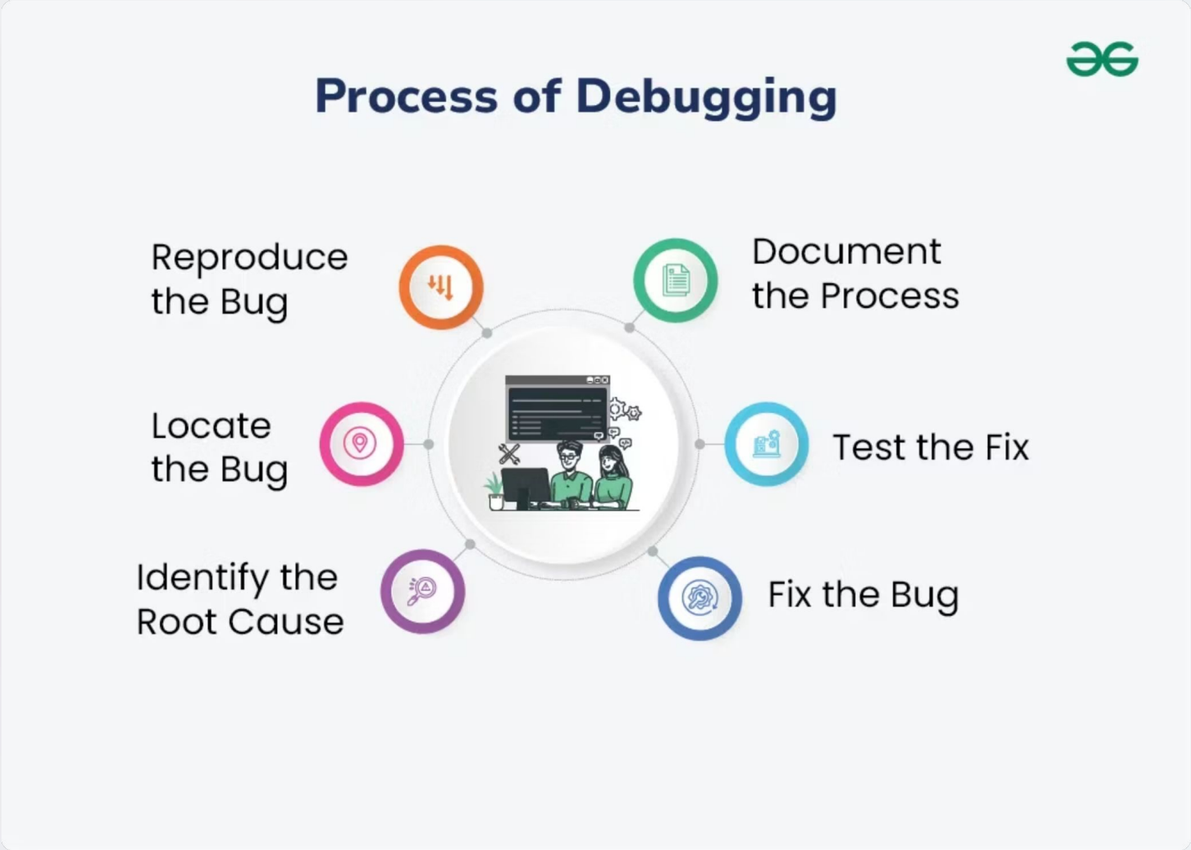
Made with GAMMA

Bug vs Error vs Debugging

Bug: incorrect behavior.

Error: runtime or compile-time message.

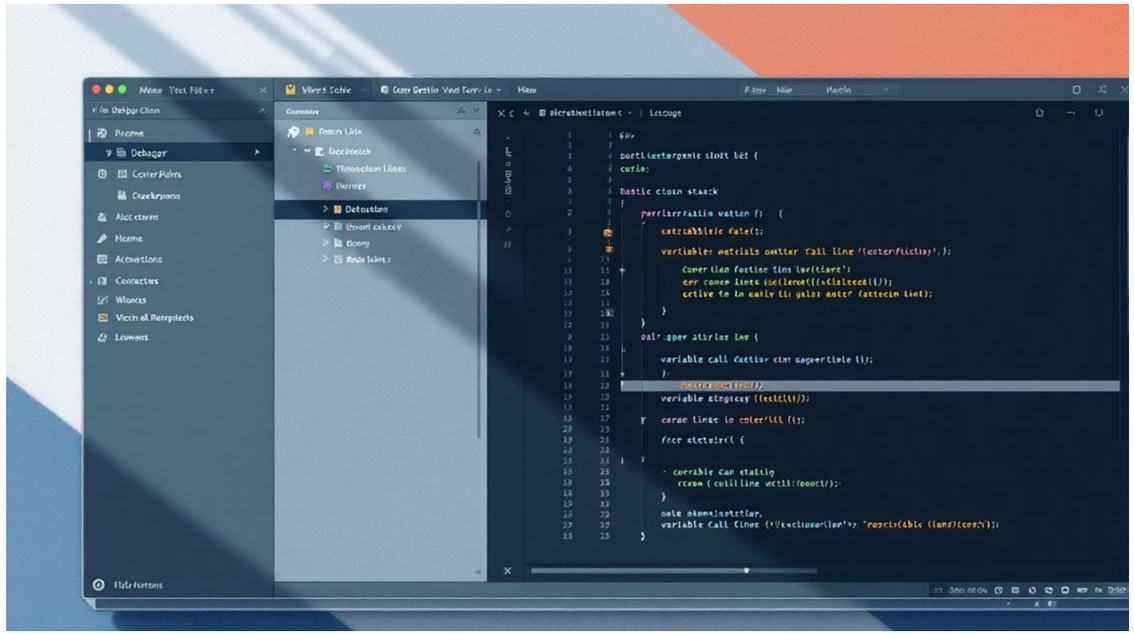
Debugging: the investigative process to resolve the bug.



Common types of Bugs

Bug	Description	Example
Functional Bug	When a feature does not work as expected.	A "Submit" button on a form doesn't perform any action when clicked
Unit-Level Bug	Bugs inside a specific function or component.	`add(2,3)` returns 6 instead of 5.
Runtime errors	Errors that occur while the program is running and often crash the program.	Dividing by zero or adding incompatible types:
Logic Bug	The program runs but produces incorrect results due to wrong logic.	A shopping cart applies the discount twice

Made with GAMMA



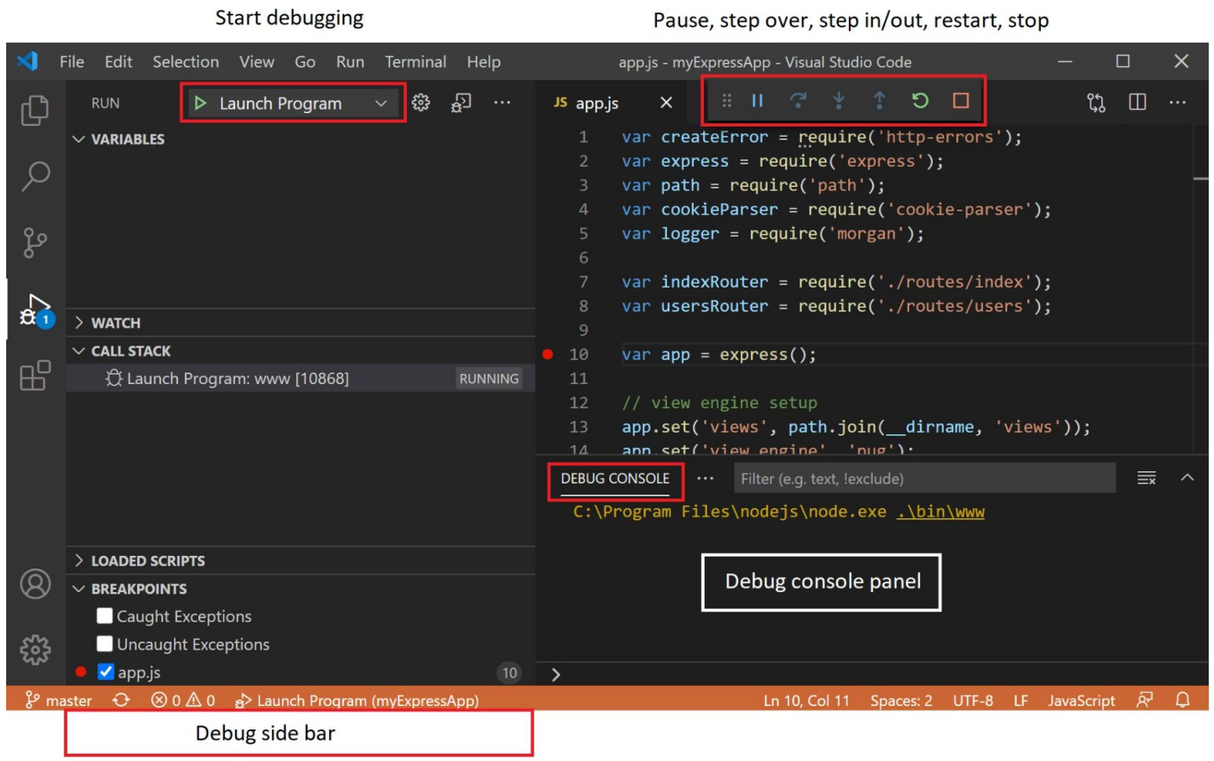
What is a Debugger?

A debugger is a tool that pauses program execution so you can inspect variables, step through code, and examine the call stack.

- Pause and resume execution
- Inspect variables & scopes
- Control execution step-by-step

Made with GAMMA

Debugger in IDE (VS code)



Run and Debug view: displays all information related to running, debugging, and managing debug configuration settings.

Debug toolbar: has buttons for the most common debugging actions.

Debug console: enables viewing and interacting with the output of your code running in the debugger.

Debug sidebar: during a debug session, lets you interact with the call stack, breakpoints, variables, and watch variables.

Run menu: has the most common run and debug commands.

Made with GAMMA

Breakpoints: Types & When to Use Them

Line Breakpoint

Standard — pause execution on a specific line to inspect state.

Conditional Breakpoint

Pause only when an expression is true (e.g., i === 1000). Useful in loops or large data sets.

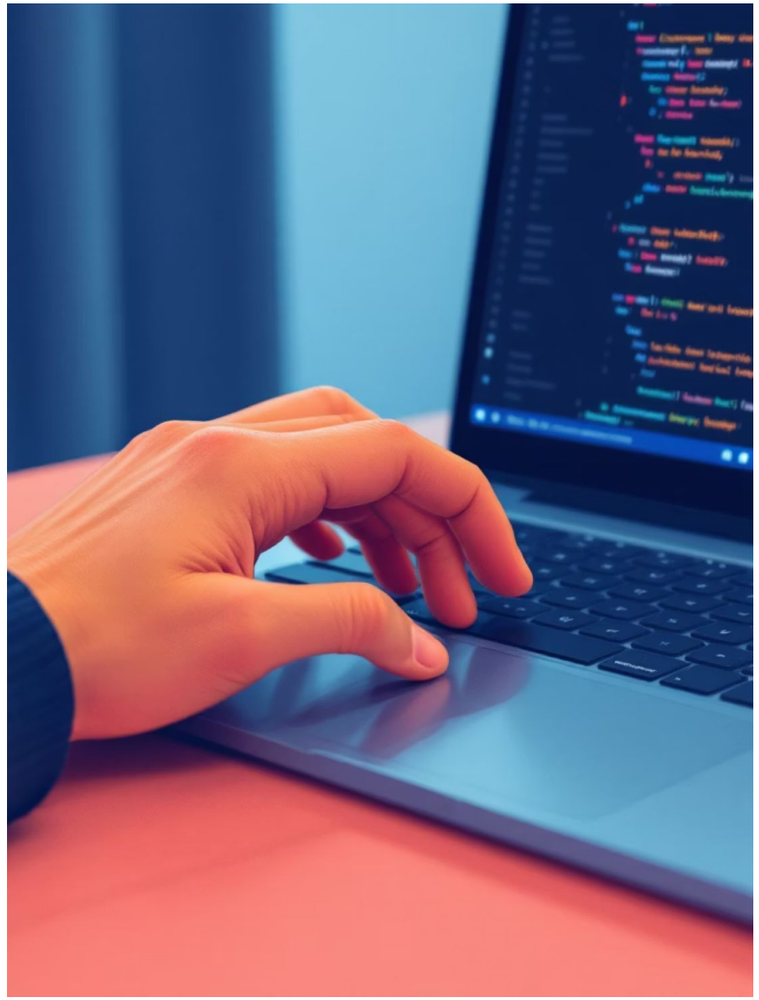
Logpoint

Emit a console message without stopping execution — good for quick tracing in production-like runs.

Use conditional breakpoints to avoid pausing on every iteration; use logpoints to add lightweight instrumentation without code changes.

Made with GAMMA

Step Controls — What Each One Does



1 **Continue**

Resume execution until next breakpoint or program exit.

2 **Step Over**

Execute the next line; if it calls a function, the function runs fully without entering it.

3 **Step Into**

Enter the called function to debug inside it line-by-line.

4 **Step Out**

Finish current function and return to the caller frame.

Made with GAMMA

Practical Example: Fixing a Type Bug in Python

Example snippet:

```
def calculate_total(price, tax):
    total = price + tax
    return total

result = calculate_total(100, "10")
print(result)
```

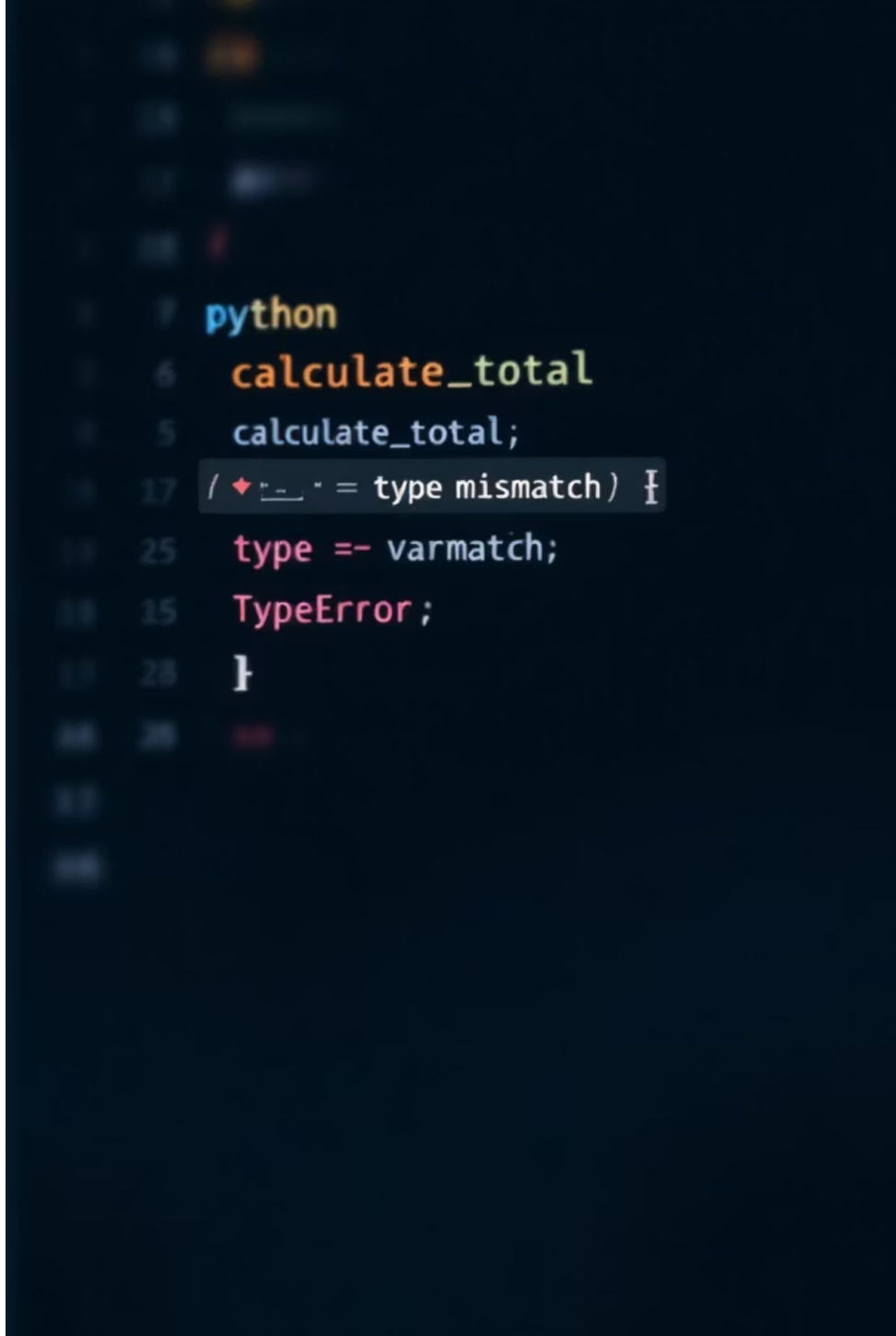
Debug steps:

1) Add a breakpoint on line returning total.

2) Run debugger → inspect types of price and tax.

3) Identify tax is a string → Step into caller to locate wrong data source.

4) Fix by converting tax to int/float or ensuring caller passes numeric value → rerun.



Reading the Evidence (Logs & Traces)

Sometimes you can't attach a debugger (e.g., in production). You must rely on files.

- **Logs:**

What they are: Text files recording events (e.g., "User logged in," "Database connected").

How to use them: Search for keywords like "Error," "Exception," or "Failed." Look at the timestamps to see the sequence of events leading to a crash.

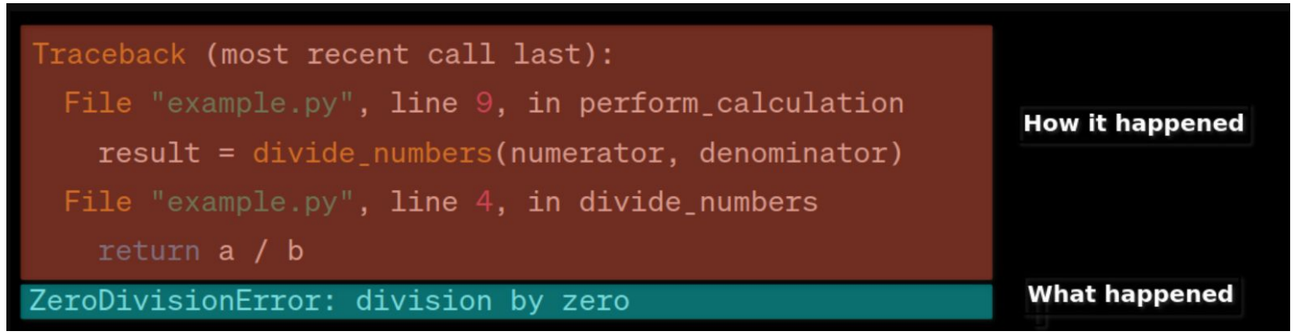
- **Error Stack (Stack Trace):**

What it is: The list of function calls that were in progress when the program crashed.

  - **How to read it:**

Read from the **bottom to top** (in python)

```
Traceback (most recent call last):
  File "example.py", line 9, in perform_calculation
    result = divide_numbers(numerator, denominator)
  File "example.py", line 4, in divide_numbers
    return a / b
ZeroDivisionError: division by zero
```



Made with GAMMA